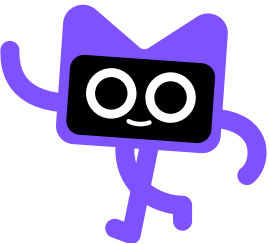
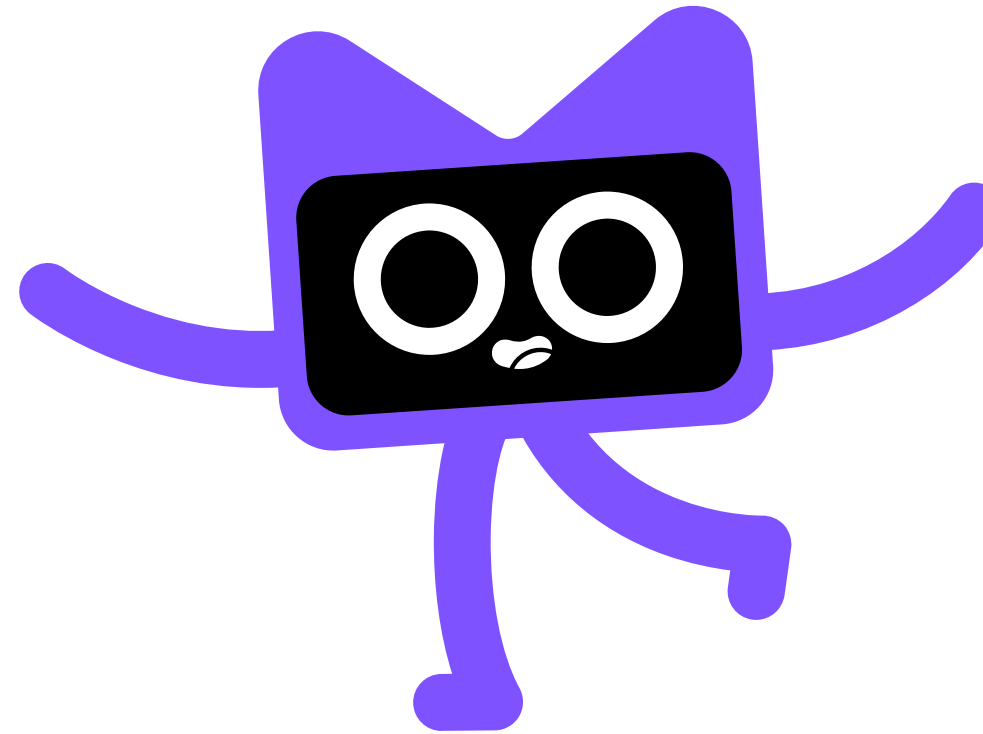


Poké-Fun with Kotlin and Arrow



Poké-Fun with Kotlin and Arrow



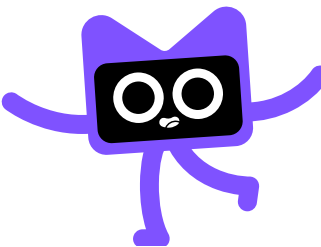
Welcome!

This is a **self-guided** workshop

1. Small introduction to Kotlin, Arrow, Compose
2. Work on the exercises you prefer

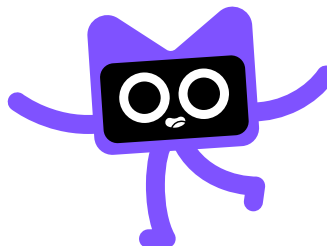
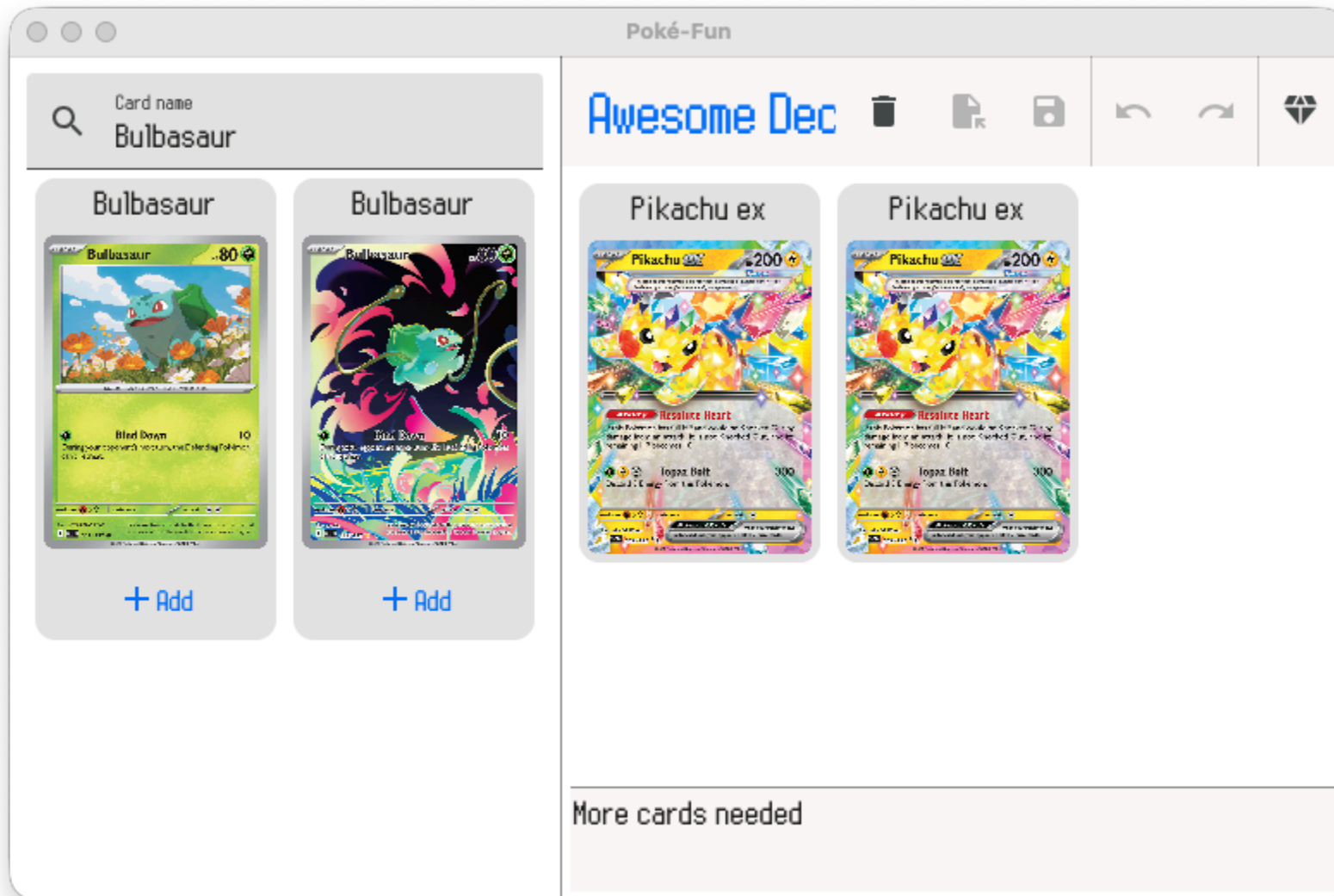
We're here to answer any question you may have 😊

| serranofp.com/poke-fun



Welcome!

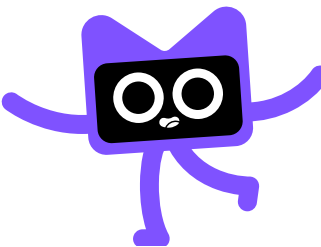
The main goal is to improve Poké-Fun, an editor for Pokémon TCG decks



Requirements

The **main page** for the workshop is `serranofp.com/poke-fun`

- IntelliJ IDEA `jetbrains.com/idea`
 - Android Studio is also an option
 - Including the Kotlin Multiplatform and Kotlin Toolchain plug-ins
- For the AI part, Ollama `ollama.com/download`
 - In macOS you can `brew install ollama`
- `git clone --recurse-submodules git@github.com:serras/poke-fun.git`



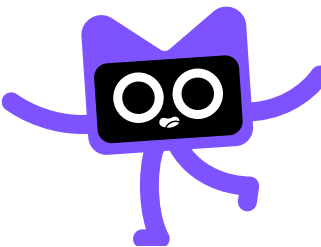
Self-guided workshop

Every chapter contains

- Some introduction to the topic at hand
- References to read / watch to learn about the topic
- Tasks to cement your understanding

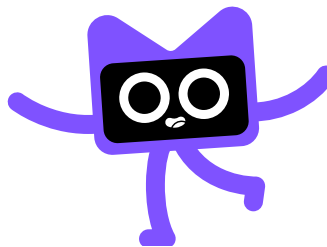
Oddish  shows you the preferred path

We're here to answer any question you may have 😊



Kotlin, Arrow, Compose 🌀

- **Kotlin** is a programming language, developed by *JetBrains*
 - Kotlin **Toolchain** is the build tool
 - `kotlinx.coroutines` is the concurrency library
- **Arrow** is a library, independently developed at `arrow-kt.io`
 - Missing utilities for functional-style programming
- **Compose** is a UI library, developed by *Google* for Android
 - Compose **Multiplatform** extends to iOS, desktop, and web

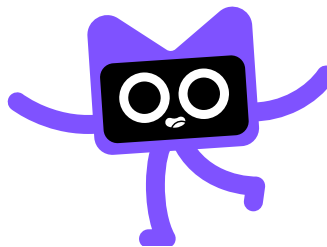


Kotlin quick intro

See kotlinlang.org/docs **to learn more**

```
data class Card(                                // data-oriented class
    val isPokemon: Boolean,                      // primary properties
    val name: String,
    val hp: Int?,                               // nullable types
) {
    val description get() = when {               // custom properties
        isPokemon → "$name ($hp HP)"           // string interpolation
        else → name
    }
}

fun Card.normalize() =                          // extension function
    Card(isPokemon, name.capitalize(), hp)      // no 'new'
```

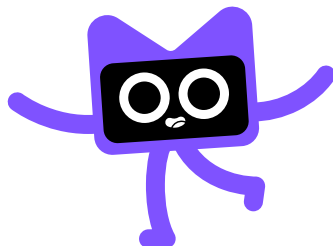


Kotlin quick intro

Great support for functional programming

- Trailing lambdas are everywhere
- Collections use `map`, `filter`, and friends

```
data class Deck(val cards: List<Card>)  
  
fun Deck.allHp(): Int =  
    cards.sumOf { card → card.hp ?: 0 }
```

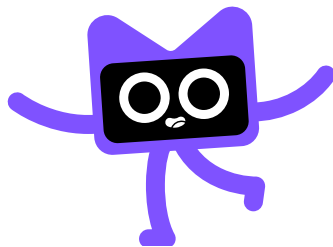


Kotlin quick intro

Great support for functional programming

- Trailing lambdas are everywhere
- Collections use `map`, `filter`, and friends

```
data class Deck(val cards: List<Card>)  
  
fun Deck.allHp(): Int =  
    cards.sumOf { it.hp ?: 0 }
```

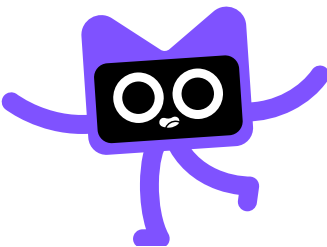


Core ideas

Value-oriented programming

Structured paradigms

Strong typing and effects



Core ideas

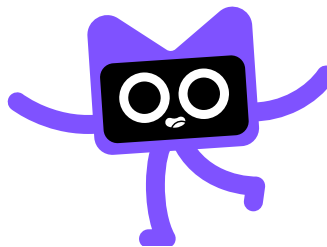
Value-oriented programming

Prefer data manipulation over complex control flow

- `Result`-like types instead of exceptions
- Work with immutable data

Structured paradigms

Strong typing and effects



Core ideas

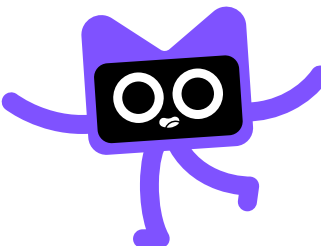
Value-oriented programming

Structured paradigms

Introduce simple structures to keep relationships between parts of the code

- Transactions that may commit or rollback
- Unidirectional flow for UIs

Strong typing and effects



Core ideas

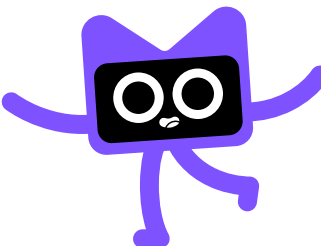
Value-oriented programming

Structured paradigms

Strong typing and effects

Keep as much information as possible in your signatures

- Give the compiler information to introduce better checks
- Explicit dependencies instead of implicit ones
- Introduce new types when new invariants pop up



Value-oriented programming

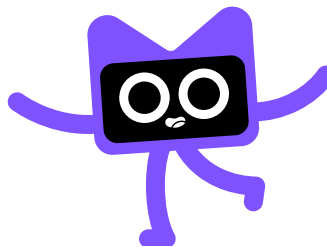
Immutable data – *What is (in) a deck*

Do not mutate data in place, always create new copies

- Simplifies the concurrency story, you never see intermediate states
- Values always preserve the invariants

Kotlin provides good support for immutability

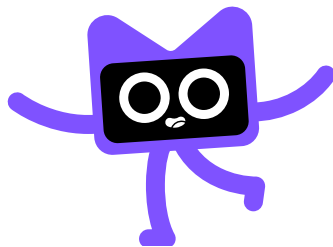
- Data classes (and soon value classes) with built-in `copy`
- Collections are immutable by default



Value-oriented programming

Immutable data – *What is (in) a deck*

```
data class Card(  
    val isPokemon: Boolean,  
    val name: String,  
    val hp: Int?,  
) {  
    val description get() = when {  
        isPokemon → "$name ($hp HP)"  
        else → name  
    }  
}  
  
fun Card.normalize() =  
    Card(isPokemon, name.capitalize(), hp)
```



Value-oriented programming

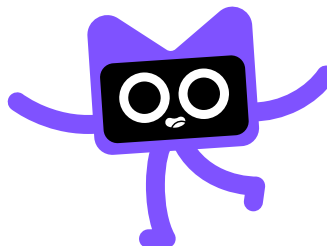
Errors – *Law-abiding decks*

Treat errors also as values, instead of special control flow

- In other words, no exceptions
- Validation becomes simpler data manipulation

```
fun pokemon(name: String, hp: Int): Either<String, Card> {  
    if (name.isEmpty()) return Left("empty name")  
    if (hp < 10)          return Left("wrong HP")  
    return Right(Card(isPokemon = true, name, hp))  
}
```

In the future, Kotlin will have *rich errors*



Value-oriented programming

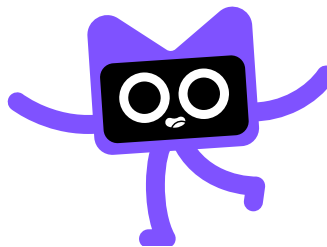
Optics – *Through the magnifying glass*

Make querying and copying immutable data much simpler

- Otherwise, immutability loses by a thousand cuts
- By default, `copy` in Kotlin is not great

Arrow Optics brings much nicer syntax

| In the future, Kotlin will have *copy syntax*



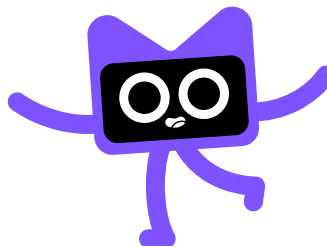
Value-oriented programming

Actions-as-data – *Deck building*

Treat descriptions of processes also as data

- Actions, events, queries... can be transformed and optimized
- Understand any program as a compiler

```
sealed interface DeckOperation {  
    data class ChangeTitle(val newTitle: String): DeckOperation  
    data class AddCard(val card: Card): DeckOperation  
    data object Clear: DeckOperation  
}
```



Value-oriented programming

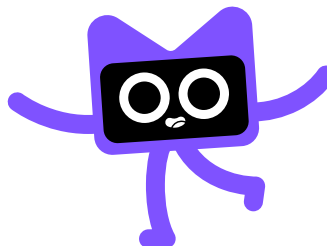
Actions-as-data – *Deck building*

Treat descriptions of processes also as data

- Actions, events, queries... can be transformed and optimized
- Understand any program as a compiler

```
fun Deck.perform(operation: DeckOperation) = when (operation) {  
    is ChangeTitle → ...  
    is AddCard → ...  
    is Clear → ...  
}
```

Works incredibly well with Compose (or React + Redux)



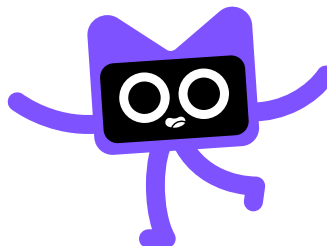
Structured paradigms

Structured concurrency

– *Loading and saving, Deal with bad internet*

Instead of creating (and then managing) independent threads, Kotlin imposes tree-like hierarchies for concurrency

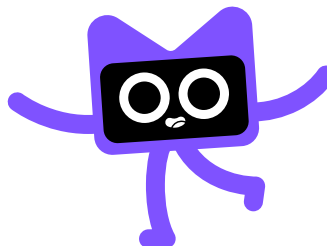
- Every *coroutine* has a parent coroutine
- By default, coroutines **wait** for all their children
- **Cancellation** is propagated to children
- **Exceptions** follow a similar propagation paradigm



Structured paradigms

Structured concurrency

```
suspend fun loadData(userId: UserId) = coroutineScope {  
    val data    = async { Db.loadUser(userId) }  
    val avatar  = async { downloadAvatar(userId) }  
    User(data.await(), avatar.await().path)  
}
```



Structured paradigms

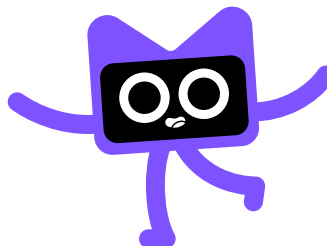
Structured concurrency

```
suspend fun loadData(userId: UserId) = coroutineScope {  
    val data    = async { Db.loadUser(userId) }  
    val avatar  = async { downloadAvatar(userId) }  
    User(data.await(), avatar.await().path)  
}
```



Coroutines are **aware of lifecycles**, such as

- Android activities (screens)
- Web server requests



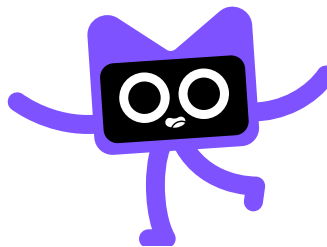
Structured paradigms

View models – *What is (in) a deck, Nicer UI*

Separate state and its updating from representation in the UI

- **View models** hold the data, usually in a flow
- **Views** define the UI and user interaction
 - In Compose, they are defined as `@Composable` functions

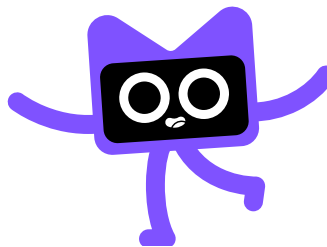
Works very well with actions-as-data, leading to **reducers**



Structured paradigms

View models – *What is (in) a deck, Nicer UI*

```
class Counter: ViewModel() {  
    // 1. define a state that evolves over time  
    val count: StateFlow<Int>  
    // 2. private state, starting at 0  
    field = MutableStateFlow(0)  
  
    // 3. operations to change the state  
    fun increment() {  
        count.update { it + 1 }  
    }  
}
```



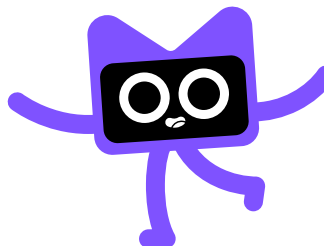
Structured paradigms

View models – *What is (in) a deck, Nicer UI*

`@Composable` functions define the UI as a tree of components

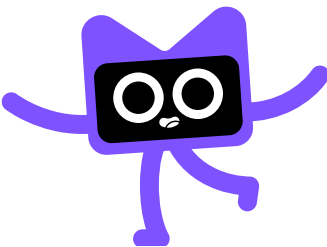
- Those components depend on the particular UI library you use

```
@Composable fun Screen(counter: Counter) {  
    Button(onClick = { counter.increment() }) {  
        val count by counter.count.collectAsState()  
        Text("Clicked $count times")  
    }  
}
```



Structured paradigms

Coroutines and **view models** are whole topics on their own, but feel free to ask about them if you want to know more

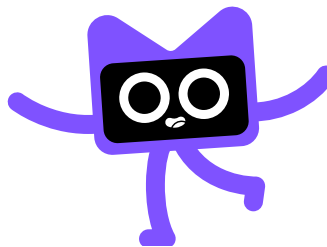


Structured paradigms

Resources and transactions – *Better architecture*

Arrow extends the idea of tree-like relationships to other architectural concerns

- **Resources** require special logic for *release*
 - On success, but also cancellation and exceptions
- **Transactions** have special *commit* requirements, and may be retried
 - Implemented using *Software Transactional Memory* (STM)
- **Sagas** have *undo* logic to perform when an error occurs



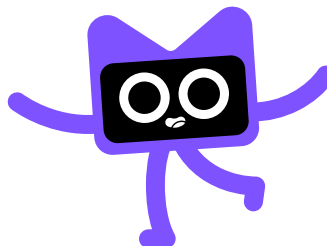
Strong typing and effects

Sealed hierarchies – *What is (in) a deck*

Closed set of classes that define all possibilities in a domain

- The compiler can check *exhaustiveness* of the code
- Introduce as many classes as needed to define your domain *precisely*

```
sealed interface DeckOperation {  
    data class ChangeTitle(val newTitle: String): DeckOperation  
    data class AddCard(val card: Card): DeckOperation  
    data object Clear: DeckOperation  
}
```



Strong typing and effects

Errors and nullability – *Law-abiding decks*

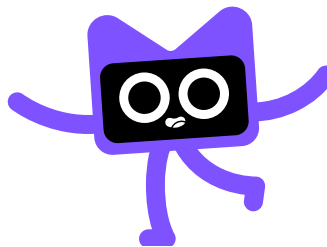
As a consequence of *strong typing* and *value-oriented programming*

Nullability is part of Kotlin types, like `Int?`

- Dedicated syntax for null-aware operations, like `x?.foo()`
- Nullability checks influence the type of the variables

Errors are also defined as potential values

- Arrow brings `Either` and `Ior` as generic wrappers



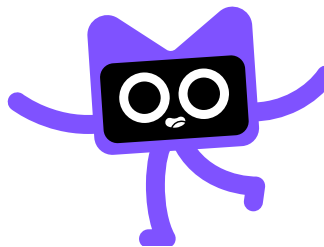
Strong typing and effects

Errors and nullability

Arrow brings **nice syntax** to define potentially-failing operations

- Very close to exception-based code, but with value-oriented semantics

```
fun pokemon(name: String, hp: Int): Either<String, Card> = either {  
    ensure(name.isNotEmpty()) { "empty name" }  
    ensure(hp ≥ 10) { "wrong HP" }  
    // if you get there, nothing was 'raise'd  
    Card(isPokemon = true, name, hp)  
}
```

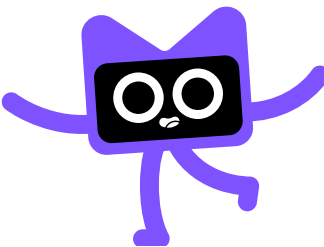


Strong typing and effects

Context parameters

Two points of view to describe what happens in the computation

- As values, with different outcomes described as a sealed hierarchy
- As **effects**, with a *context* that defines additional *capabilities*



Strong typing and effects

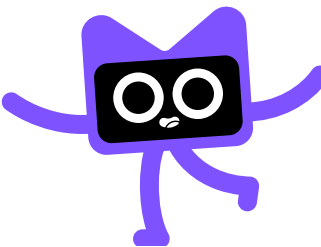
Context parameters

Two points of view to describe what happens in the computation

- As values, with different outcomes described as a sealed hierarchy
- As **effects**, with a *context* that defines additional *capabilities*

Examples of effects

- Whether the computation may have a failure path
- Whether the computation uses concurrency
- Whether talking to a database is required



Strong typing and effects

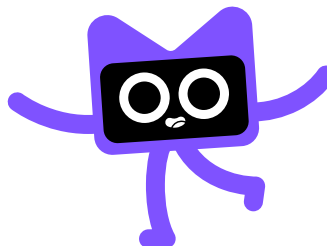
Context parameters

Two points of view to describe what happens in the computation

- As values, with different outcomes described as a sealed hierarchy
- As **effects**, with a *context* that defines additional *capabilities*

Examples of effects

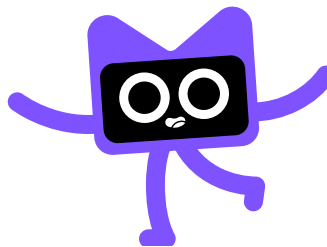
- Whether the computation may have a failure path $\Rightarrow$ `Raise`
- Whether the computation uses concurrency $\Rightarrow$ `CoroutineScope`
- Whether talking to a database is required $\Rightarrow$ `DbService`



Strong typing and effects

Context parameters

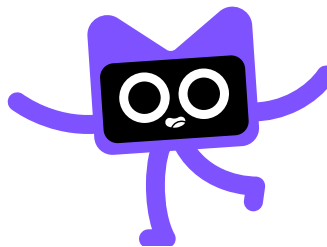
```
fun pokemon(name: String, hp: Int): Either<String, Card> = either {  
    ensure(name.isNotEmpty()) { "empty name" }  
    ensure(hp ≥ 10) { "wrong HP" }  
    // if you get there, nothing was 'raise'd  
    Card(isPokemon = true, name, hp)  
}
```



Strong typing and effects

Context parameters

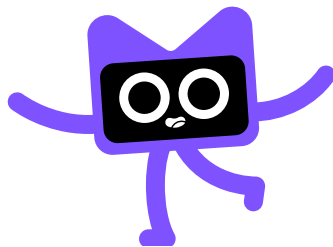
```
context(raise: Raise<String>)
fun pokemon(name: String, hp: Int): Card {
    ensure(name.isNotEmpty()) { "empty name" }
    ensure(hp ≥ 10) { "wrong HP" }
    // if you get there, nothing was 'raise'd
    return Card(isPokemon = true, name, hp)
}
```



AI enters the room

The workshop contains a few sections about using LLMs

- Using local models thanks to **Ollama**
- With **Koog** as the library of choice



Time to work!

serranofp.com/poke-fun

